

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



LARGE ASSIGNMENT
MACHINE LEARNING
WORD-SENSE DISAMBIGUATION

CLASS CC01 --- SEM 251

Instructor: Dr. NGUYỄN ĐỨC DŨNG

Student	ID
Trần Thái Tài	2353065
Lưu Tôn Thịnh	2353139
Ngô Vĩnh Quang	2352968

Ho Chi Minh City – January 2026

TABLE OF CONTENT

1. Problem Statement and Motivation	1
2. Existing Solutions	1
3. Our Approach.....	2
4. Demonstration and Experiments	4
5. Deep Learning vs. Classic Machine Learning	5
6. Selection of the Best Classic ML Approach	6
Source Code: https://github.com/ZenTonia2k5/Word-sense-Disambiguation-ML-and-DL.....	6
REFERENCES	7

1. Problem Statement and Motivation

Natural language is inherently ambiguous, presenting a fundamental challenge in the field of Natural Language Processing (NLP). This ambiguity is best exemplified by the phenomenon of Polysemy, where a single word form possesses multiple distinct meanings depending on its usage context. For instance, the word "bank" generally refers to a financial institution, yet in a topological context, it refers to the land alongside a river. Similarly, "charge" can imply a financial transaction, a criminal accusation, or an electrical property. Word-Sense Disambiguation (WSD) is the computational task of resolving this ambiguity by identifying which specific sense of a word is intended in a given sentence. The goal is to classify a target word into one of its pre-defined senses derived from a lexicon.

Our motivation for investigating this problem stems from its role as a critical prerequisite for advanced Artificial Intelligence applications. We recognize that without effective WSD, downstream tasks suffer from significant semantic errors. In Machine Translation, for example, a system must distinguish between "crane" as a bird and "crane" as a construction machine to generate the correct translation in a target language. Furthermore, accurate Information Retrieval relies on understanding user intent to separate search results for "jaguar" the animal from "jaguar" the vehicle. By addressing WSD, we aim to contribute to systems that move beyond simple pattern matching toward true language understanding.

2. Existing Solutions

Research into WSD reveals a historical progression from knowledge-based rules to modern data-driven approaches. Initially, the field relied on Knowledge-Based Methods such as the Lesk Algorithm. This unsupervised approach utilizes external dictionaries and assumes that words in a given neighborhood share a common topic, determining the correct sense by measuring the overlap between the dictionary definition of the target word and the surrounding context words.

As the availability of annotated data increased, the field shifted toward Supervised Machine Learning. These methods treat WSD as a classification task trained on sense-annotated corpora. Classic Machine Learning algorithms, including Naive Bayes, Decision Trees, and Support Vector Machines (SVM), typically utilize feature engineering techniques like "bag-of-words" and Part-of-Speech tagging. More recently, the field has been dominated by Deep Learning approaches. Modern architectures utilizing Recurrent Neural Networks (RNNs) and Transformers (such as BERT) have replaced manual feature extraction with dense vector embeddings, allowing models to capture complex semantic relationships and long-distance dependencies that traditional methods often miss.

3. Our Approach

In this project, we implemented and compared three distinct models to solve the WSD problem: a probabilistic Naive Bayes classifier, a distance-based k-Nearest Neighbors (k-NN) classifier, and a Deep Learning model based on Long Short-Term

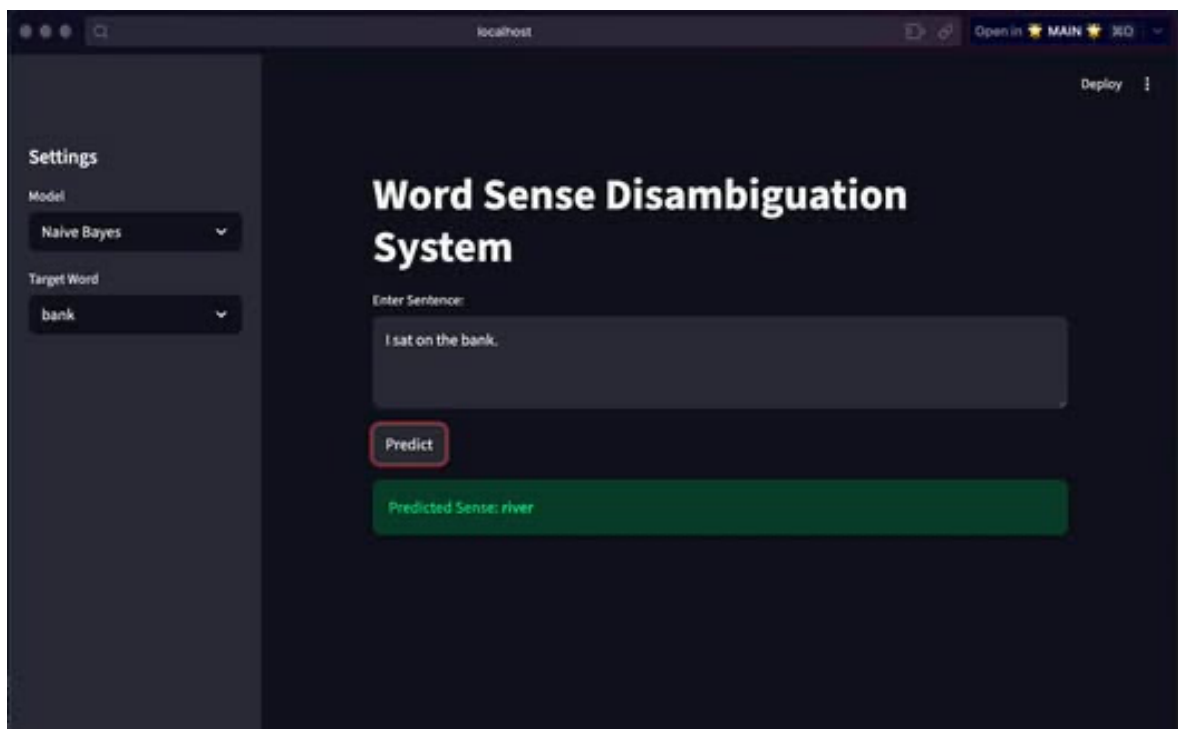


Figure 1: Demo of Naive Bayes

Memory (LSTM) networks. Our implementation leverages Python, utilizing the Scikit-learn library for the classic algorithms and PyTorch for the deep learning component. We also developed an interactive web interface using Streamlit to demonstrate our models in real-time.

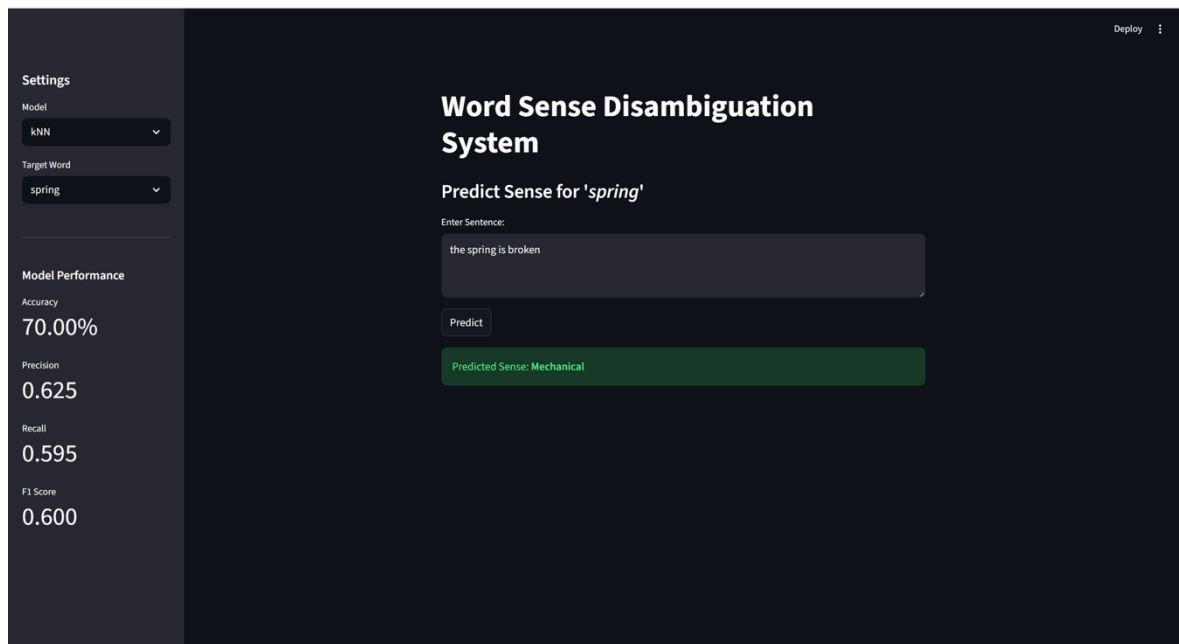


Figure 2: k-NN Model

Our methodology began with data preprocessing, where we utilized custom datasets containing sentences paired with their specific sense labels for target words such as "bank," "charge," "light," "match," and "spring." For the classic models, we employed a CountVectorizer to convert text into numerical feature vectors. The Naive Bayes model was implemented to calculate the conditional probability of each sense given the context words, applying Laplace smoothing to handle unseen vocabulary. The k-NN model utilized cosine similarity to find the nearest labeled examples in the vector space.

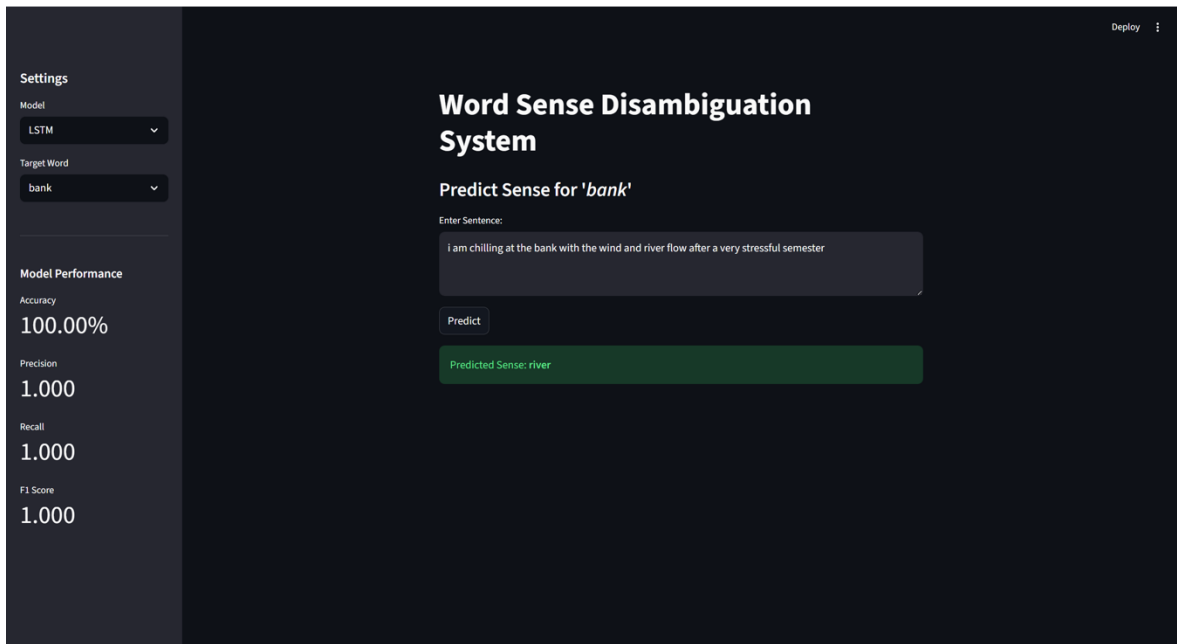


Figure 3: LTSM

For the Deep Learning approach, we constructed a Bidirectional LSTM (Bi-LSTM). Unlike the classic models that treat sentences as unordered bags of words, our LSTM preserves the sequence of the input. We employed an embedding layer to represent words as dense vectors and trained the network to capture context from both past and future words in the sentence. The final output is passed through a fully connected linear layer to predict the correct sense class.

4. Demonstration and Experiments

We validated our approach by training our models on the provided datasets and evaluating them on unseen test sentences. Our experiments focused on the ability of each model to distinguish between subtle context shifts. We measured performance using standard metrics including Accuracy, Precision, Recall, and F1-score.

Qualitative testing demonstrated the strengths of our approach. For example, when processing the sentence "I went to the bank to deposit money," our system correctly identified the financial institution sense due to the strong association with words like

"deposit" and "money." Similarly, for the input "The boat drifted towards the river bank," the models successfully shifted the prediction to the river-side sense. We observed that while the Naive Bayes model performed reliably on sentences containing strong keywords, the LSTM model demonstrated a superior ability to handle sentences where context was defined by sentence structure rather than specific keywords. We have hosted our code and a demonstration of these experiments on our GitHub repository.

5. Deep Learning vs. Classic Machine Learning

Our investigation highlights significant differences between Deep Learning and Classic Machine Learning in the context of WSD. The primary distinction lies in contextual representation. Classic approaches like Naive Bayes and k-NN rely heavily on the presence of specific words within a fixed window. They treat words as independent features, often failing to capture the order of words or the nuance of synonyms. If a test sentence uses vocabulary that does not overlap with the training data, these models frequently fail.

In contrast, Deep Learning models like our Bi-LSTM utilize continuous vector space representations. This allows them to capture semantic similarities between words even if they are not identical. Furthermore, the recurrent nature of the LSTM allows it to maintain a memory of the entire sentence structure, understanding that a word appearing at the beginning of a sentence can influence the meaning of a target word at the end. While the Deep Learning approach yields significantly higher accuracy and generalization, we noted it requires substantially more computational resources and training data compared to the lightweight and fast execution of the classic algorithms.

6. Selection of the Best Classic ML Approach

If we were restricted to choosing a single Classic Machine Learning approach as the most appropriate for WSD, we would select the Support Vector Machine (SVM). Although our project implemented Naive Bayes and k-NN for comparison, theoretical and historical evidence suggests SVM is superior for this specific domain.

The rationale for this choice is threefold. First, NLP tasks inherently involve massive feature spaces with vocabularies spanning thousands of words. SVMs are mathematically designed to be robust in such high-dimensional spaces, resisting the overfitting that plagues algorithms like Decision Trees. Second, text data is highly sparse, meaning most words do not appear in any single sentence. Linear SVMs are exceptionally efficient at finding optimal hyperplanes to separate classes in sparse datasets. Finally, before the advent of Deep Learning, SVMs consistently achieved state-of-the-art performance in international WSD competitions, proving their reliability as the gold standard of the classic era.

Source Code:

<https://github.com/ZenTonia2k5/Word-sense-Disambiguation-ML-and-DL>

REFERENCES

1. Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297. <https://doi.org/10.1007/BF00994018>
2. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)* (pp. 4171–4186). Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423>
3. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
4. Jurafsky, D., & Martin, J. H. (2024). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition* (3rd ed. draft). <https://web.stanford.edu/~jurafsky/slp3/>
5. Lesk, M. (1986). Automatic sense disambiguation using machine readable dictionaries: How to tell a pine cone from an ice cream cone. In *Proceedings of the 5th Annual International Conference on Systems Documentation* (pp. 24–26). Association for Computing Machinery. <https://doi.org/10.1145/318723.318728>
6. Navigli, R. (2009). Word sense disambiguation: A survey. *ACM Computing Surveys*, 41(2), 1–69. <https://doi.org/10.1145/1459352.1459355>
7. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, & E. Fox (Eds.), *Advances in Neural Information Processing Systems* 32 (pp. 8024–8035). Curran Associates, Inc.

8. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
9. Streamlit Inc. (2024). *Streamlit* (Version 1.32.0) [Computer software]. <https://streamlit.io/>